Lookup Table Convolutional Spiking Neural Network - LUTCSNN

Sauram Vasanawala, Kyrus Reyaz Mama, Kwabena Boahen,
Department of Bioengineering, Stanford University

Abstract

The energy-efficiency of the brain vastly outperforms that of current AI models. Biological neurons only spike if their dendrites receive signal in a specific order, therefore, signaling can be sparse [1]. Lookup tables (LUTs) replicate this behavior of neurons by consisting of many parameters but using few at a time. Izhikevich (2025) harnessed LUTs to create a spiking neural network (SNN) which learns parameters using surrogate gradient descent [2]. This LUTSNN architecture has been used to reproduce multilayer perceptrons, recurrent neural networks, and transformers, but has yet to be used to reproduce convolutional neural networks (CNNs), the cornerstone of computer vision. Here, we redesign the LUT architecture to mirror convolution layers, enabling table sharing and accumulative gradient updates. We also rework dropout regularization and pooling components to improve performance. Finally, we dive into the trade-off between increasing the total number of parameters and decreasing gradient signal in backwards passes.

Motivation: Making NNs Sparse

Artificial neural networks (ANNs) have all neurons fire at once using all parameters each forward pass, relying on large matrix multiplication and interpreting inputs as continuous activations (Fig. 1). LUTSNNs interpret inputs as spike-timings allowing the firing of few neurons selective to the specific spike order. Thus, LUTSNNs use a fraction of the total parameters each forward pass and eliminate matrix multiplication completely. This sparse use of parameters makes LUTSNNs potentially more efficient than ANNs. The extension of LUTs to CNNs would have countless applications to computer vision.

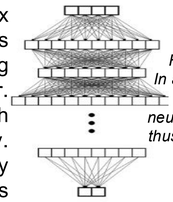


Figure 1: In an ANN, every neuron fires thus it lacks sparsity

Background: LUT Architecture

Forward Pass (Fig. 2)
- Input: Vector of latency of spikes (x)
- Each table
  - Makes nc comparisons: $u_i = x_{a_i} - x_{b_i}$ where $i = 1, \dots, \#\ of\ comparisons\ (nc)$
  - Row index: $j = concat(u_1 > 0, \dots, u_{nc} > 0)$
  - Output: row j from table S, a new latency vector S_j
- Latency output vectors from each table are summed and passed to next layer

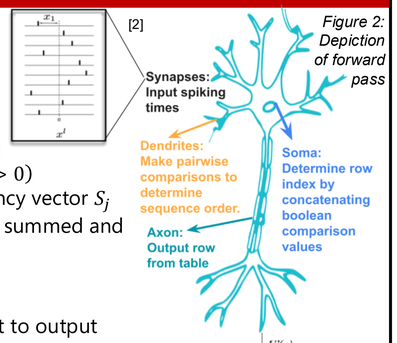


Figure 2: Depiction of forward pass

Backward Pass
- Input: Change in loss function with respect to output
- Update table: $S\ -= learning\ rate \cdot \frac{\partial L}{\partial x^{l+1}}$
- Surrogate gradient using uncertainty function (Fig. 3) to estimate continuity of table rows
- Pick only most uncertain comparison to compute gradient
- Chain rule: $\frac{\partial L}{\partial x_m^l} = \frac{\partial L}{\partial x^{l+1}} \cdot \frac{\partial x^{l+1}}{\partial x_m^l}$
- Surrogate gradient: $\frac{\partial x^{l+1}}{\partial x_a} = -\frac{\partial x^{l+1}}{\partial x_b} = U'(x_a - x_b) \cdot (S_{\vec{x}} - S_x)$
- Output: $\frac{\partial L}{\partial x_a^l} = -\frac{\partial L}{\partial x_b^l} = \frac{\partial L}{\partial x^{l+1}} \cdot U'(x_a^l - x_b^l) \cdot (S_{\vec{x}^l}^l - S_{x^l}^l)$
- Output passed to the previous layer's backward as input

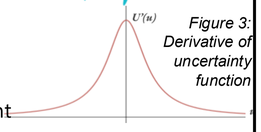


Figure 3: Derivative of uncertainty function

LUTCSNN Design

Traditional convolution layers convolve a filter with each patch in an image to get an output channel (Fig. 4).

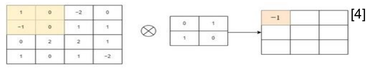


Figure 4: Application of regular convolution filter to a patch

In an LUT convolution layer, we replace the convolving with LUTs.
Forward Pass:
- For each patch apply LUT (Fig. 5), anchors move relative to patch center
- Each convolution filter uses a specific set of tables
- Each output channel is the sum of multiple tables' outputs

Backward Pass
- Same as normal LUT, map gradient back to correct pixels in correct patches

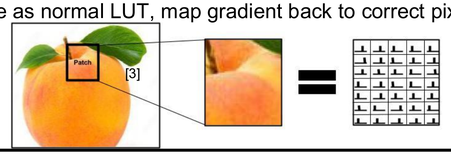


Figure 5: Depiction of patch's conversion to spike timings in preparation to apply LUT

Dropout Regularization Design

- Expected to prevent overfitting by stopping the model from relying too heavily on individual tables or comparisons
- Randomly set tables to zero
- Tables don't contribute to the sum in forward pass (Fig. 6)
- Excluded tables also excluded in backward pass

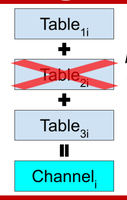


Figure 6: Diagram of table dropout

Pooling Design

- Pooling in regular CNNs used for subsampling and to decrease spatial size
- Expected that only mean pooling will work for LUT design because output is discontinuous unlike regular CNNs

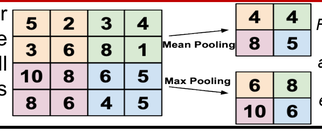


Figure 7: Mean and max pooling example

Number of Comparisons Theory

- Expected number of output pixels impacting a table row in a backward pass: $\frac{Total\ \#\ of\ Output\ Pixels}{2^{Number\ of\ Comparisons}}$
- Surrogate gradient is noisy
- We need more updates to average out noise of gradient

Increase number of comparisons
- Decrease number of updates to each row per backward pass
- Decrease signal to noise ratio (SNR) of gradient (Fig. 9)

$SNR = (\frac{mean\ of\ gradient\ on\ class\ i\ images}{std\ of\ gradient\ on\ class\ i\ images})^2$

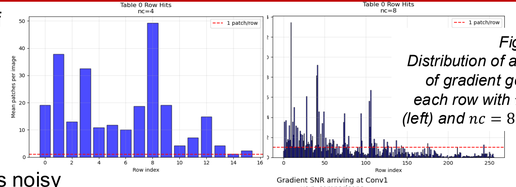


Figure 8: Distribution of amount of gradient going to each row with $nc = 4$ (left) and $nc = 8$ (right)

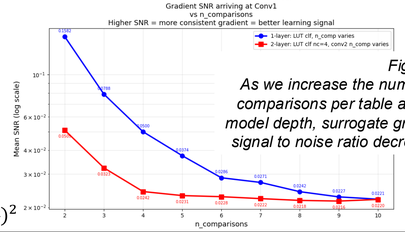


Figure 9: As we increase the number of comparisons per table and the model depth, surrogate gradient signal to noise ratio decreases.

LUTCSNN Results

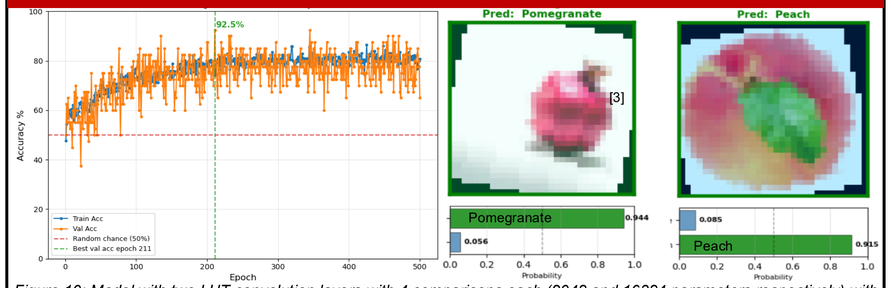
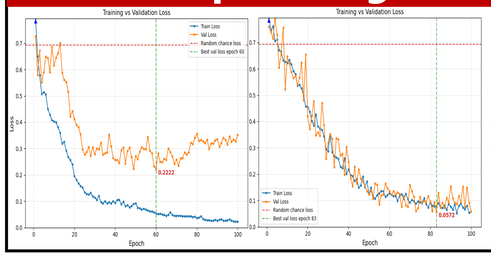


Figure 10: Model with two LUT convolution layers with 4 comparisons each (2048 and 16384 parameters respectively) with LUT linear layer with 4 comparisons (32768 parameters) (left) performs similarly to regular CNN with two convolution layers (432 and 4608 parameters respectively) and one linear layer (12288 parameters). Each LUTSCNN forward pass uses a fourth of the total parameters used by the CNN. Correct prediction on pomegranate (middle) and peach (right).

Dropout Regularization Results



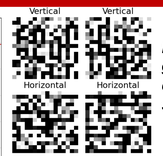


Figure 11: Two class line dataset generated using noise added to either vertical or horizontal striped lines.

Figure 12: Model with two LUT convolution layer and one LUT linear layer trained on line data without table dropout (left) and with table dropout of 0.2 in all three layers (right). The addition of table dropout prevented overfitting on training data.

Pooling Results

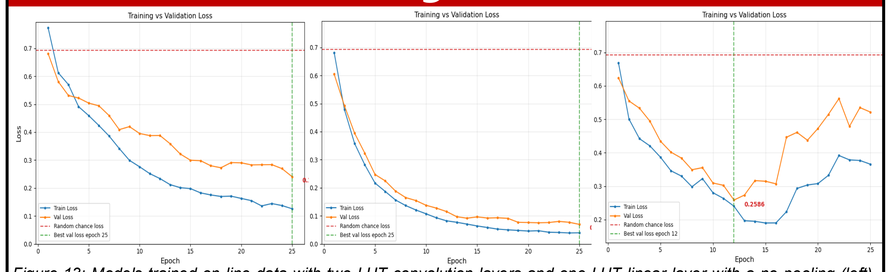


Figure 13: Models trained on line data with two LUT convolution layers and one LUT linear layer with a no pooling (left), mean pooling (middle), and max pooling (right) layer after each conv layer. Mean pooling model performed the best.

Future Work

- Use LUT CSNN to do image segmentation, video analysis, and generation
- Flesh out theory regarding surrogate gradient signal strength

[1] Boahen, K. Dendrocentric learning for synthetic intelligence. *Nature* **612**, 43–50 (2022). https://doi.org/10.1038/s41586-022-05340-6
[2] Izhikevich, E. (2025). Spiking manifesto. *arXiv preprint arXiv:2512.11843*.
[3] GTS, GTS (2025), "Fruits Dataset for Classification", Mendeley Data, V1, doi: 10.17632/rg254yr63x.1
[4] Taye, M. M. (2023). Theoretical Understanding of Convolutional Neural Network: Concepts, Architectures, Applications, Future Directions. *Computation*, *11*(3), 52. https://doi.org/10.3390/computation11030052